

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 5+6+9

Date : 08/11/2023
Matière : Programmation OO Java
Durée : 1h
Documents : Non autorisés
Nombre des pages : 2

Devoir Surveillé

Exercice 1

Qu'affiche le programme suivant ?

```
public class Exercice1{
    static{
        System.out.println("Message");
    }
    public static void main(String[] args) {
        int a=10;
        int b=011;
        int c=-3;
        System.out.println("b0 = "+b);
        System.out.println("Opération sur c = "+~c);
        var result = switch (a > b ? "grand" : "petit") {
            case "grand" -> ++a-++c+b++;
            case "petit" -> {
                int val = ++b*a--;
                yield (val+c);
            }
            default -> ++b;
        };
        System.out.println("Résultat = "+result);
        System.out.println("b1 = "+b);
        var op = (b != 10) && (a/b < 1) ? 1 : 2;
        switch (op) {
            case 0:
                System.out.println("Test 0");
                break;
            case 1:
                System.out.println("Test 1");
            case 2:
                System.out.println("Test 2");
            default:
                System.out.println("Test");
                break;
        }
    }
}
```

Exercice 2

Écrivez un programme Java qui prend trois nombres réels (x, y et z) en ligne de commande et détermine le maximum et le minimum de ces trois nombres **sans utiliser** les méthodes max et min de la classe Math.

Exercice 3

1. Écrivez une classe **Point** qui représente un point dans un espace bidimensionnel. Cette classe doit comporter les attributs et les méthodes suivants :
 - Attributs :
 - double x : représentant l'abscisse du point.
 - double y : représentant l'ordonnée du point.
 - Un constructeur prenant deux paramètres pour initialiser les coordonnées du point.
 - Méthodes :
 - double getX() : renvoyant la coordonnée x du point.
 - double getY() : renvoyant la coordonnée y du point.
 - double distanceTo(Point otherPoint) : calculant la distance euclidienne entre le point courant et un autre point donné en tant que paramètre.
 - void translate(double dx, double dy) : effectuant une translation du point en déplaçant ses coordonnées de dx sur les abscisses et de dy sur les ordonnées.
 - void homothetie(double scaleFactor) : réalisant une homothétie du point par rapport à l'origine en multipliant ses coordonnées par un facteur d'échelle spécifié.
 - static Point barycentre(Point[] points) : calculant et renvoyant les coordonnées du barycentre des points spécifiés dans le tableau. Le barycentre est calculé comme étant la moyenne des coordonnées de ces points.
2. Écrivez une classe **Segment** qui modélise un segment de ligne entre deux points. Cette classe doit comporter les attributs et les méthodes suivants :
 - Attributs :
 - Point point1 : représentant le premier point d'extrémité du segment.
 - Point point2 : représentant le deuxième point d'extrémité du segment.
 - Un constructeur prenant deux références de type Point en tant que points d'extrémité du segment.
 - Méthodes :
 - Point getPoint1() : renvoyant le premier point d'extrémité du segment.
 - Point getPoint2() : renvoyant le deuxième point d'extrémité du segment.
 - double length() : calculant et renvoyant la longueur du segment, c'est-à-dire la distance entre les deux points d'extrémité.

Remarques :

- Respectez le **principe d'encapsulation** en déclarant les attributs et les méthodes.
- La distance euclidienne entre deux points (x1, y1) et (x2, y2) est égale à :

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

- La classe **Java.lang.Math** contient deux méthodes statiques permettant de calculer la racine carrée et la puissance d'un certain nombre.
Voici la déclaration de ces méthodes :

```
class Math {  
    public static double sqrt(double nb) :      // retourne la racine carrée de nb  
    public static double pow(double nb, double p) : // retourne nbp  
    ...  
}
```

BON TRAVAIL